Bölüm 1: Python Temelleri

SE 422 · Introduction to Data Science · Konu özeti

Kaynak slayt destesi: Week2_1
7 Kasım 2026

İçindekiler

Bölüm 1 · Python Temelleri 3

Bölümün amacı 3

Konu anlatımı 3

Yorumlayıcı ve iki çalışma modu (Slayt 5–8) 3

String'ler ve yorumlar (Slayt 9–10) 3

Değişkenler ve atama (Slayt 11–15) 4

Veri tipleri (Slayt 16–17) 4

Klavyeden girdi ve tip dönüştürme (Slayt 18–19) 5

Matematik operatörleri ve öncelik (Slayt 20–23) 5

Karışık tipler ve uzun satırlar (Slayt 24–26) 6

String birleştirme ve print ayrıntıları (Slayt 27–32) 6

f-string ile biçimli çıktı (Slayt 33–40) 7

Sihirli sayılar ve isimli sabitler (Slayt 41–44) 7

if ve if-else ile karar verme (Slayt 45–57) 8

String karşılaştırma (Slayt 58–59) 8

Mantıksal operatörler, kısa devre ve Boolean değişkenler (Slayt 60–71) 8

Sık yapılan hatalar 9

Kısa tekrar soruları 10

Cevaplar 10

Bölüm 1 · Python Temelleri

Bölümün amacı

Bu bölüm bir Python programının temel döngüsünü öğretir: **girdi al → işle → çıktı ver**. Sonunda şunları yapabilmelisin:

- Python yorumlayıcısını etkileşimli modda ve betik modunda kullanmak
- String, yorum ve değişkenlerle çalışmak; veri tiplerini (int, float, str) tanımak
- Klavyeden girdi almak ve sayıya dönüştürmek, matematik operatörleriyle hesap yapmak
- `print` ve f-string ile biçimli çıktı üretmek
- İsimli sabitler, `if` / `if-else` ile karar vermek; ilişkisel ve mantıksal operatörleri kullanmak

Konu anlatımı

Yorumlayıcı ve iki çalışma modu (Slayt 5–8)

Python kodunu okuyup çalıştıran programa **yorumlayıcı** (interpreter) denir. İki şekilde kullanılır:

- **Etkileşimli mod:** komutları tek tek yazarsın; `>>>` istemi yorumlayıcının komut beklediğini gösterir. Yazılanlar kaydedilmez, denemeler için idealdir.
- **Betik modu:** komutlar `.py` uzantılı bir dosyaya yazılır ve komut satırında `python dosyaadi.py` ile çalıştırılır.

IDLE, Python ile birlikte kurulan; yazma, çalıştırma ve test araçlarını bir araya getiren, etkileşimli modda çalışan ve yerleşik metin editörü olan bir geliştirme ortamıdır.

String'ler ve yorumlar (Slayt 9–10)

String, veri olarak kullanılan karakter dizisidir. Kodda doğrudan yazılan string'e **string literal** denir; tek ya da çift tırnak içine yazılır. Üç tırnak (`'''` veya `"""`) hem iki tırnak türünü içerebilir hem de birden çok satıra yayılabilir.

`#` ile başlayan **yorumlar** insan okuyucu içindir; yorumlayıcı onları yok sayar.

```python
print("Kate'in kitabı")  # çift tırnak içinde tek tırnak kullanılabilir
print('''Birinci satır
İkinci satır''')
```

ÇIKTI

```
Kate'in kitabı
Birinci satır
İkinci satır
```

Değişkenler ve atama (Slayt 11–15)

Değişken, bellekteki bir değeri temsil eden isimdir. Atama ifadesiyle oluşturulur: `degisken = ifade`. Değeri alan değişken her zaman **solda** olur (`29 = age` hatadır). Değişken adı tırnak içine yazılmaz: `print('age')` değişkeni değil, "age" kelimesini yazar.

İsimlendirme kuralları: anahtar kelime olamaz, boşluk içeremez, harf veya _ ile başlar, büyük/küçük harfe duyarlıdır (`sayi` ≠ `Sayi`). Ad, değişkenin amacını anlatmalıdır (`x` yerine `saatlik_ucret`).

`print` birden çok öğeyi virgülle alır ve aralarına boşluk koyar. Bir değişkene sonradan yeni değer atanabilir; artık hiçbir değişkenin göstermediği değerler **çöp toplama** (garbage collection) ile bellekten silinir.

PYTHON

```python
age = 29
print(age)
print('age')
fiyat = 10
print('Fiyat:', fiyat, 'TL')
fiyat = 12
print('Yeni fiyat:', fiyat)
```

ÇIKTI

```
29
age
Fiyat: 10 TL
Yeni fiyat: 12
```

Veri tipleri (Slayt 16–17)

Değerler tiplere ayrılır: tam sayı **int**, ondalıklı sayı **float**, metin **str**. Ondalık noktası olmayan sayısal literal int (`7`), olan float'tır (`7.0`). Python'da bir değişken önce sayıyı, sonra metni gösterebilir.

PYTHON

```python
x = 99
print(type(x))
x = 'Take me to your leader'
print(type(x))
```

PYTHON (DEVAM)

```
print(type(7.0))
```

ÇIKTI

```
<class 'int'>
<class 'str'>
<class 'float'>
```

Klavyeden girdi ve tip dönüştürme (Slayt 18–19)

`input(istem)` klavyeden okur ve **her zaman string döndürür**. Sayı gerekiyorsa `int()` veya `float()` ile dönüştürülür; metin geçerli bir sayı değilse istisna (exception) oluşur.

PYTHON

```
metin = '42'          # input() bir string döndürmüş gibi düşün
sayi = int(metin)
print(sayi + 8)
print(float('3.5') * 2)
```

ÇIKTI

```
50
7.0
```

PYTHON

```
print(int('kırk iki'))
```

ÇIKTI

```
ValueError: invalid literal for int() with base 10: 'kırk iki'
```

Matematik operatörleri ve öncelik (Slayt 20–23)

Operatör	Anlamı	Örnek
`+ - *`	toplama, çıkarma, çarpma	`2 * 3` → 6
`/`	ondalıklı bölme	`7 / 2` → 3.5
`//`	tam sayı bölmesi	`7 // 2` → 3
`%`	bölümden kalan	`17 % 5` → 2
`**`	üs	`2 ** 3` → 8

Öncelik (yüksekten düşüğe): parantez → `**` → `* / // %` → `+ -`; aynı öncelikte soldan sağa. `//` pozitif sonucu keser, negatif sonucu sıfırdan uzağa yuvarlar.

PYTHON
```
print(7 / 2, 7 // 2, -7 // 2)
print(2 + 3 * 4, (2 + 3) * 4)
print(2 ** 3, 17 % 5)
```

ÇIKTI
```
3.5 3 -4
14 20
8 2
```

`%` tek/çift bulmada ve zaman dönüştürmede işe yarar:

PYTHON
```
saniye = 3725
print(saniye // 3600, 'saat', saniye % 3600 // 60, 'dakika', saniye % 60,
↪ 'saniye')
print(14 % 2 == 0)
```

ÇIKTI
```
1 saat 2 dakika 5 saniye
True
```

Karışık tipler ve uzun satırlar (Slayt 24–26)

İki int → int, iki float → float, karışık → float. `int(3.9)` ondalık kısmı **keser** (yuvarlamaz). Uzun ifadeler `\` ile ya da parantez içinde satırlara bölünebilir.

PYTHON
```
print(5 + 2.0)
print(int(3.9))
toplam = (1 + 2 +
          3 + 4)
print(toplam)
```

ÇIKTI
```
7.0
3
10
```

String birleştirme ve print ayrıntıları (Slayt 27–32)

`+` iki string'i birleştirir. Yan yana yazılan string literal'ler **örtük olarak** birleşir. `print` sonuna yeni satır koyar; `end=` bunu, `sep=` ise öğeler arasındaki boşluğu değiştirir. `\n` yeni satır, `\t` sekme karakteridir.

PYTHON
```python
print('Hello ' + 'world')
print('one' 'two' 'three')
print('A', 'B', 'C', sep='-')
print('Bir', end=' ')
print('İki')
print('Satır1\nSatır2\tsekmeli')
```

ÇIKTI
```
Hello world
onetwothree
A-B-C
Bir İki
Satır1
Satır2    sekmeli
```

f-string ile biçimli çıktı (Slayt 33–40)

Başına `f` konan string'de `{ }` yer tutucuları değişken ya da ifade alır. İki noktadan sonra **biçim belirteci** yazılır: sıra `[hizalama][genişlik][,][.hassasiyet][tip]`.

PYTHON
```python
name = 'Johnny'
num = 12345.6789
print(f'Hello {name}. The value is {10 + 2}.')
print(f'{num:.2f} | {num:,.2f} | {num:.2e}')
print(f'{0.5:.0%} | {123456789:,d}')
print(f'[{num:>12,.2f}] [{num:^12.1f}] [{num:<12.0f}]')
```

ÇIKTI
```
Hello Johnny. The value is 12.
12345.68 | 12,345.68 | 1.23e+04
50% | 123,456,789
[   12,345.68] [  12345.7   ] [12346       ]
```

Sihirli sayılar ve isimli sabitler (Slayt 41–44)

Kodda açıklamasız geçen sayılar (**sihirli sayı**) okunmayı zorlaştırır, değiştirmeyi zahmetli ve hataya açık yapar. Çözüm: büyük harfle yazılan **isimli sabit**.

PYTHON
```python
INTEREST_RATE = 0.069
balance = 1000
amount = balance * INTEREST_RATE
print(f'Faiz: {amount:.2f}')
```

ÇIKTI

```
Faiz: 69.00
```

if ve if-else ile karar verme (Slayt 45–57)

Sıra yapısında ifadeler sırayla çalışır; **karar yapısında** bazıları yalnızca bir koşul doğruysa çalışır. `if koşul:` satırının altındaki **girintili blok** koşul `True` ise çalışır. `else:` bloğu ise koşul yanlışken çalışır; `if` ve `else` aynı hizada yazılır.

Koşullar **ilişkisel operatörlerle** kurulur: `>`, `<`, `>=`, `<=`, `==` (eşit mi?), `!=` (eşit değil mi?). `=` atama, `==` karşılaştırmadır.

PYTHON

```python
sales = 60000
bonus = 500.0 if sales > 50000 else 0
temperature = 35
if temperature < 40:
    print("A little cold, isn't it?")
else:
    print("Nice weather we're having.")
print(bonus, 5 == 5, 5 != 5, 4 <= 5)
```

ÇIKTI

```
A little cold, isn't it?
500.0 True False True
```

String karşılaştırma (Slayt 58–59)

String'ler `==`, `!=`, `<`, `>` ile karşılaştırılabilir. Karşılaştırma **büyük/küçük harfe duyarlıdır** ve karakter karakter, karakter kodlarına göre yapılır. Kısa kelime uzun kelimenin başıysa uzun olan büyüktür.

PYTHON

```python
print('Mary' > 'Mark', 'a' == 'A', 'Ali' < 'Alice')
print(ord('y'), ord('k'))
```

ÇIKTI

```
True False True
121 107
```

Mantıksal operatörler, kısa devre ve Boolean değişkenler (Slayt 60–71)

- `and`: iki taraf da doğruysa True · `or`: biri doğruysa True · `not`: değeri ters çevirir.
- **Kısa devre:** `or`'da sol taraf True ise, `and`'de sol taraf False ise sağ taraf hiç değerlendirilmez.

- Aralık içi: `x >= 10 and x <= 20` · aralık dışı: `x < 10 or x > 20`.
- `in` / `not in` bir değerin bir dizide (ör. string) olup olmadığını sınar.
- **Boolean değişken** True/False tutar ve genellikle **bayrak** olarak kullanılır.

PYTHON
```
un, pwd = 'arda', '12345'
print(un == 'arda' and pwd == '12345')
hour, temp = 14, 28
print(hour < 15 or temp >= 28, not (hour <= 14))
x = 25
print(x >= 10 and x <= 20, x < 10 or x > 20)
print('s' in 'sebastian', '123' not in 'Password:12345')
```

ÇIKTI
```
True
True False
False True
True False
```

Sık yapılan hatalar

- **Değeri atanmadan değişkeni kullanmak** → `NameError`. Ayrıca `age` ile `Age` farklı değişkenlerdir.
- **input'un sonucunu sayı sanmak:** `input()` string döndürür; `'5' + 1` hata verir, önce `int()` gerekir.
- **String'e sayı eklemek:** `'Yaş: ' + 20` → `TypeError`. `str(20)` ya da f-string kullan.

PYTHON
```
yas = 20
print('Yaşın: ' + yas)
```

ÇIKTI
```
TypeError: can only concatenate str (not "int") to str
```

- `if` **satırının sonundaki** `:` **işaretini unutmak** veya `=` **ile** `==`**'yi karıştırmak** → `SyntaxError`.
- `int(3.9)`**'un yuvarladığını sanmak:** keser, sonuç 3'tür.
- `and`**'in iki tarafını tam yazmamak:** `x >= 10 and <= 20` geçersizdir; `x >= 10 and x <= 20` yazılır.

Kısa tekrar soruları

1. `print(7 // 2, -7 // 2, 7 % 2)` ne yazdırır?
2. `input()` hangi tipte değer döndürür? Bir sayıyla toplamak için ne yapmalısın?
3. `f'{1234.5:,.1f}'` ifadesinin sonucu nedir?
4. Sihirli sayı nedir? Neden isimli sabit tercih edilir?
5. `'apple' < 'apricot'` neden True'dur?
6. `x = 5` iken `x > 3 or 1 / 0 > 1` hata verir mi? Neden?

Cevaplar

1. `3 -4 1` (tam sayı bölmesi negatifte sıfırdan uzağa yuvarlar).
2. Her zaman `str`. Önce `int()` veya `float()` ile dönüştürmelisin.
3. `1,234.5`
4. Kodda açıklamasız geçen sayıdır. İsimli sabit kodu kendini açıklar hale getirir, değeri tek yerden değiştirmeyi sağlar ve yazım hatasını önler.
5. Karşılaştırma karakter karakter yapılır: `ap` aynı, üçüncü karakterde `p` (112) < `r` (114).
6. Vermez. `or`'da sol taraf True olduğu için kısa devre olur, `1 / 0` hiç çalışmaz.